



Creando Oportunidades

ETHER SOFTWARE DEVELOPMENT PLATFORM

Manual de usuario v0.1

A&GD

Madrid, Marzo 2020

SOBRE ESTE MANUAL	4
Objetivo	4
INTRODUCCIÓN	5
PLATFORM OVERVIEW	5
Conceptos	5
Producto	5
Aplicación	5
Recursos técnicos	5
Roles	6
Procesos	7
Global and regional resources	8
Ether Cloud Services - Conceptos básicos	9
Tools	10
CONSOLA	10
ECS CLI	10
GETTING STARTED	12
Starting by programming language or runtime	12
ASO-LINK	12
APX	12
CELLS	12
DATAPROC	12
APÉNDICE - Uso de ECS-CLI y Stacker para creación de recursos y despliegues.	12
Requisitos previos - Instalación ECS-CLI	13
Requisitos previos - Instalación ECS-CLI	13
Creación de una versión	17
Despliegue de una versión en un namespace (entorno)	17
Services	21
ECS Stacker	21
¿Qué es?	21
¿En qué consiste la “creación de nuevos recursos”?	21
¿En qué consiste el “despliegue de recursos”?	22
Técnicamente, ¿de qué partes se compone Stacker?	23

SOBRE ESTE MANUAL

Objetivo

Este manual debe servir de base para informar y explicar a cualquier nuevo usuario de la plataforma, los conceptos básicos organizativos, técnicos, de proceso, procedimiento y normativos necesarios para construir, desplegar y operar software en **ETHER PLATFORM**.

ETHER SOFTWARE DEVELOPMENT PLATFORM es el subconjunto de servicios, herramientas, procesos y normas para la construcción de software que se desplegará sobre la plataforma **ETHER**.

1. INTRODUCCIÓN

Esta descripción general está diseñada para ayudarle a comprender el panorama general de **ETHER SOFTWARE DEVELOPMENT PLATFORM**. Aquí, podrá ver algunas de las funciones de uso común y obtendrá enlaces a la documentación específica que puede ayudarle a profundizar. Saber qué hay disponible y cómo funcionan las partes juntas puede ayudarle a tomar decisiones sobre cómo proceder. También obtendrá enlaces a algunos tutoriales que puede usar para probar **ESDP** en varios escenarios.

2. PLATFORM OVERVIEW

2.1. Conceptos

El concepto funcional base dentro de la ESDP es el producto. Cada producto podrá agrupar un conjunto de aplicaciones, y cada aplicación se compone de un conjunto de recursos.

Producto

Un “producto” es el término que se utiliza para referirse a un conjunto de recursos técnicos y aplicaciones que resuelven un problema o prestan un servicio al cliente. Este producto tiene un ciclo de vida, unos actores

Aplicación

El concepto de una aplicación Ether permite agrupar varios recursos técnicos como servicios ASO, transacciones APX y componentes CELLS.

Las aplicaciones tienen relación N a 1 con las UUAA, es decir, cada aplicación pertenece a una UUAA, y una UUAA puede tener varias aplicaciones.

Recursos técnicos

En Ether, las aplicaciones se estructuran en componentes técnicos o módulos que vamos a llamar recursos. Según la necesidad de cada aplicación, se podrán añadir uno o varios recursos de diferentes tipos. Una aplicación tendrá 1 a n recursos

La plataforma ofrece un amplio catálogo de recursos basados en los servicios que ofrece Ether, como pueden ser, por ejemplo, componentes de *frontend* (cells, salesforce), *banking runtimes* (pylls, apx, aso-link, lra, ...), procesos *batch* y *datastores*.

Por el momento los únicos recursos disponibles son:

- Banking runtimes:
 - APX
 - BPM
 - ASO
 - CELLS
 - DataProc
 - Pylls

Roles Ether

Los Roles Ether sirven para determinar las responsabilidades y acciones técnicas a ser desempeñadas por un conjunto de personas que trabajan dentro de una UA de tipo NextGen. Ej Desarrolladores, Technical leads.

Estos roles se aplican a dos niveles distintos: a nivel de cada UA individual o transversal a una determinada área organizativa.

Para acotar el conjunto de personas que forman parte de un Rol se han creado los Grupos de Consola Ether (que a su vez están persistidos en GIAM como servicio normativo dentro del Banco para la gestión de identidades y autorizaciones).

Los Grupos de Consola Ether pueden ser de dos tipos:

- Cross: cubren funciones cross, especialmente enfocados a dar soporte, a todas las aplicaciones y productos a lo largo del ciclo de vida del software. ej: Support Group (que habilitan el acceso a múltiples UUAA dentro de un ámbito geográfico y/o de UUAA que hacen uso de un determinado runtime/arquitectura.
- UA Groups/Custom Groups: que se crean programáticamente asociados a cada UA y cubren funciones para esa UA específica. Ej. Desarrolladores de la UA KGPE Globales.

Dentro de la plataforma existen distintos roles con diferentes permisos dentro del ciclo de vida del desarrollo de un producto:

01	Solution Architect	- Responsible de crear las aplicaciones/UAs, después de definir la solución funcional. Configura las geografías asociadas a cada aplicación e indica las listas de tecnologías necesarias para desarrollar los recursos. - Tiene permisos de administración relativos a asignar/dar de alta a otros usuarios que tendrán el rol de tech leader de la UA.
02	Technical leader	- Es el rol responsable de la UA, y puede haber uno/s específicos para cada una de las geografías que participan en la UA. - Tiene permisos de administración, normalmente se encargará de dar de alta a los demás usuarios de su equipo con su rol/permisos adecuados. - Responsible de la gestión del código fuente, su implantación y mantenimiento/soporte en cada una de las geografías.
03	Release Manager	- Responsible de las versiones de la aplicación. - Tiene permisos para la creación y despliegue de versiones, tanto en entornos previos como el entorno de Producción, para la geografía de la UA en la que tenga asignado este rol. También puede autorizar las solicitudes de despliegues en Producción enviadas por los Developers de la UA. - Puede realizar Rollbacks y borrado de despliegues en cualquiera de los entornos (para los runtimes que tienen implementada dicha funcionalidad)
04	Developer	- Se encarga del desarrollo/programación de los recursos técnicos de la aplicación. - Puede acceder a la información de la UA, acceder a los repositorios de la UA, versionar los componentes, desplegarlos en entornos previos y solicitar despliegues en Producción. - Hay que tener en cuenta el matiz de que en las UUAA de tipo, implementación Global y despliegue MultiLocal, los permisos del rol Developer varían en función de si la

persona está asignada en la geografía Global (donde el Developer tiene todos los permisos a la hora de crear componentes técnicos y de edición del código en los repositorios), de si el Developer está incluido en alguna de las geografías de despliegue (donde su función y permisos, respecto a la edición del código será sólo para poder crear hotfix, y los tendrá para poder desplegar los componentes sólo en los entornos de su geografía).

- Puede realizar Rollbacks y borrado de despliegues en los entornos previos (para los runtimes que tienen implementada dicha funcionalidad)
-

Hay que aclarar que un usuario puede adoptar distintos roles al mismo tiempo, y que los roles aún no siendo literalmente jerárquicos (el hecho de tener un rol no te da permisos que no estén recogidos por ese rol), pero sí presentan cierta jerarquía.

Technical Lead > Release Manager > Developer

- El rol **Developer** puede realizar principalmente las funciones especificadas anteriormente
- El **Release Manager** puede hacer las funciones del Developer más algunas específicas (desplegar en Producción y autorizar Despliegues en Producción)
- El **Technical Lead** puede realizar las tareas habilitadas al Release Manager más algunas funciones específicas de su rol como:
 - Habilitar nuevas geografías y entornos para la UA
 - Si el TL es de la geografía de implementación, gestionar determinadas TechSpecs de la UA (que son las tecnologías que se quieren habilitar para poder ser usadas a la hora de implementar los componentes técnicos de la UA)
 - Gestionar los miembros del Team de la UA así como la asignación del rol que le quiere aplicar a cada una de las personas del equipo.

Procesos

Un proceso es un conjunto de tareas secuenciadas que deben ser completadas por los usuarios y/o sistemas implicados para alcanzar con éxito una funcionalidad con peso en la organización.

Se han definido un conjunto de procesos que nos sirven para gestionar el ciclo de vida de los recursos desde el punto de vista del Alta y Modificación.

En los siguientes enlaces se da acceso de forma conjunta a la información de soporte a los procesos :

- Diagrama de proceso donde se visualizan las tareas, los roles que las acometen y transiciones entre las misas
- Documento con el detalle del proceso donde se explica cada tarea en particular.

Para recursos ASO-Link, los enlaces son:

[Procesos Servicios ASO](#)

- [Detalle Proceso Alta Servicios ASO](#)
- [Detalle Proceso Modificación Servicios ASO](#)
- [Detalle Proceso Modificación Express Servicio ASO](#)
- [Detalle Proceso Solicitud Consumo](#)

Para recursos APX, los enlaces son:

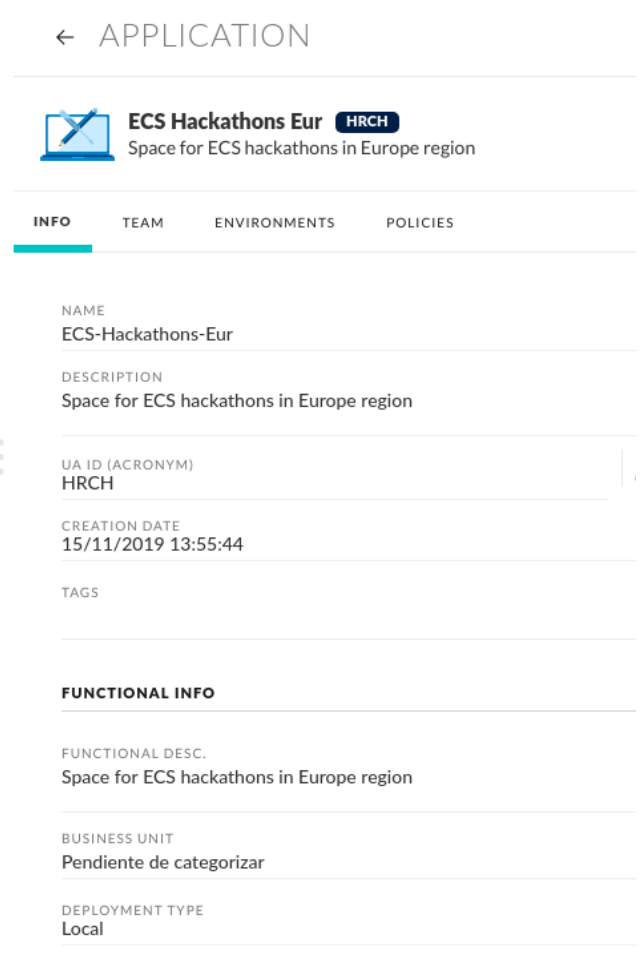
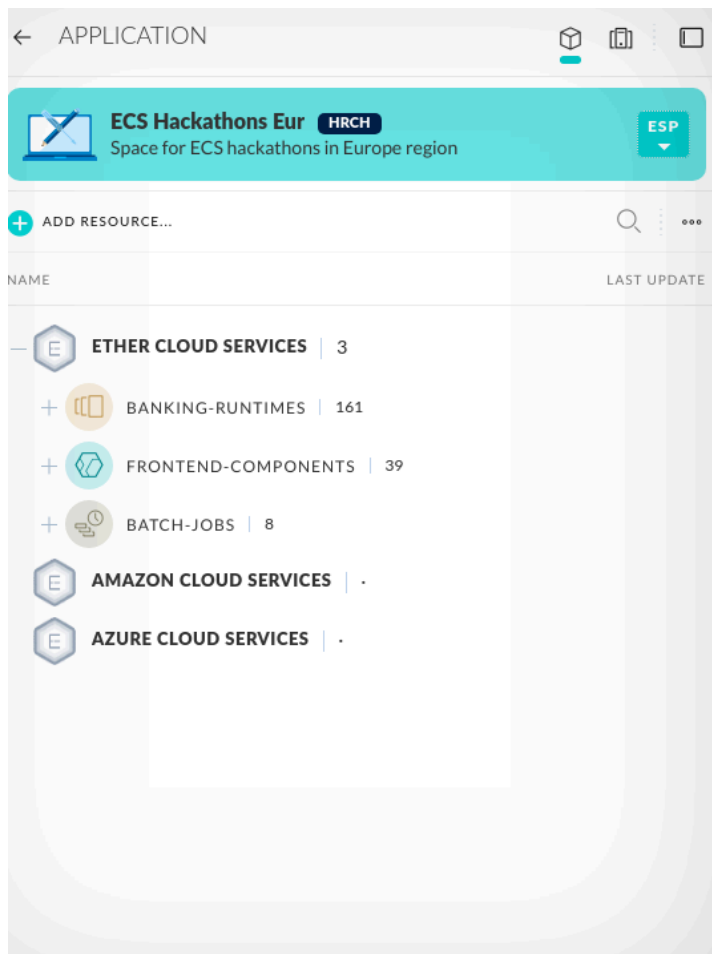
[Procesos Componentes APX](#)

- [Detalle Proceso Alta Componentes APX](#)
- [Detalle Proceso Modificación Componentes APX](#)
- [Detalle Proceso Modificación Express Componentes APX](#)

Indicar que el detalle de las tareas a abordar en cada proceso se encontrará en los manuales específicos de los runtimes.

Global and regional resources

Cuando se accede a una aplicación, si esta se ha creado para múltiples geografías, se gestionan los recursos para cada una de ellas por separado. Como se puede observar en la siguiente imagen, al lado del título de la aplicación, aparece un combo con las posibles regiones. Al cambiar la región se recargan los recursos y la información asociada ya que pueden ser diferentes dependiendo de la geografía.



Ether Cloud Services - Conceptos básicos

Ether Cloud Services (ECS) es la nube privada utilizada para diseñar y desplegar los recursos. Información detallada de ECS, incluyendo los servicios disponibles, se puede consultar en <https://ecs.bbva.com>.

A modo de resumen, se describen los principales conceptos de ECS:

- Región. Siguiendo un enfoque similar a las regiones de las cloud públicas, definen zonas donde se encuentran disponibles los servicios ofrecidos por ECS. Actualmente tenemos las siguientes regiones:
 - central: En esta región se encuentran desplegados los servicios encargados de dar soporte al ciclo de vida de desarrollo como son los de DevOps. Los namespaces de diseño, los cuales incluyen la definición de los recursos así como sus repositorios y jobs de construcción asociados, se crean en esta región. Los namespaces de diseño de las soluciones de todas las geografías (globales o locales) se crean en esta región.

- work-01. Región de baja latencia en Europa con datos NO reales. Se utiliza para las soluciones de global, España y Turquía. Por cada entorno lógico de las geografías anteriores que usa datos no reales, se crea un namespace en esta región.
- live-01. Región de baja latencia en Europa con datos reales. Se utiliza para las soluciones de global, España y Turquía. Por cada entorno lógico (productivo o previo) de las geografías anteriores que usa datos reales, se crea un namespace en esta región.
- work-02. Región de baja latencia en América con datos NO reales. Se utiliza para las soluciones de los países pertenecientes a América. Por cada entorno lógico de las geografías anteriores que usa datos no reales, se crea un namespace en esta región.
- live-02. Región de baja latencia en América con datos reales. Se utiliza para las soluciones de los países pertenecientes a América. Por cada entorno lógico (productivo o previo) de las geografías anteriores que usa datos reales, se crea un namespace en esta región.
- Namespace. Es una partición lógica de la nube similar a las cuentas en el caso de AWS o los proyectos en el caso de GCP. De modo automático, por cada aplicación se crean los siguientes namespaces:
 - Diseño. Se crea un namespace de diseño por cada aplicación/geografía en la que se van a desarrollar recursos. Por ejemplo, si se trata de una solución de desarrollo global y despliegue multilocal, se crea un único namespace de diseño para la aplicación. Si en un desarrollo multilocal, se crea un namespace de diseño por cada geografía asociados a la misma aplicación. Si se trata de un desarrollo puramente local, se crea un único namespace de diseño correspondiente a esa aplicación/geografía. Este tipo de namespaces se crean siempre en la región central.
 - Ejecución. Son los namespaces que representan entornos previos o productivos donde se ejecuta la aplicación. Por cada aplicación/geografía se crean namespaces en dos regiones:
 - **Entornos previos** (work-01 / work-02). Por cada entorno previo se crea un namespace en work-01 o work-02 dependiendo de la geografía a la que pertenezca.
 - **Entornos con datos reales** (live-01 / live-02). Por cada entorno con datos reales (por ejemplo producción o piloto) se crea un namespace en live-01 o live-02 dependiendo de la geografía a la que pertenezca.

2.2. Tools

CONSOLA

La consola ETHER Es la herramienta principal desde la que los Sol. Architects, Technical Leads, Developers y Release Managers podrán aprovisionar, configurar, desplegar y operar sus aplicaciones

Ether cumpliendo de una forma sencilla y amigable los procedimientos y procesos de gobierno de BBVA.

ECS CLI

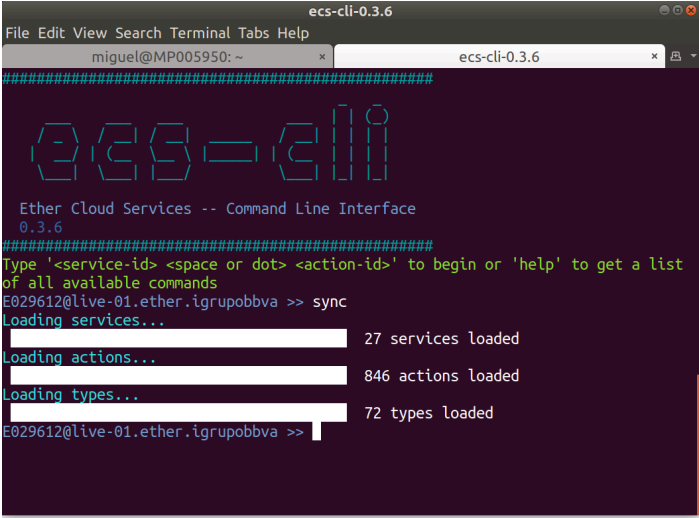
[ECS CLI](#) es una herramienta unificada de línea de comandos para interactuar con los servicios y recursos de ECS. Entre otros servicios y de modo complementario a la consola Ether, permite invocar a ECS Stacker para crear nuevos recursos, registrar nuevas versiones de los mismos y desplegarlos.

La instalación de ECS CLI no requiere permisos de administración. Las instrucciones así como los ejecutables para cada plataforma están disponibles en [ECS CLI Installation](#)

La herramienta encargada de proporcionar los tokens de autenticación para uso de ECS-CLI es Portunus la cual está disponible en "<https://portunus.live-01.nextgen.igrupobbva/>". Más información relativa a la gestión de credenciales en ECS disponible en [ECS Credentials](#)

Al final de este documento, a modo de apéndice, se describen los pasos necesarios para crear recursos con stacker y desplegarlos con ECS-CLI.

[Descripcion herramientas GDT - equipo tools y testing]



```
ecs-cli-0.3.6
File Edit View Search Terminal Tabs Help
miguel@MP005950: ~ x ecs-cli-0.3.6 x
#####
  ECS-CLI
Ether Cloud Services -- Command Line Interface
0.3.6
#####
Type '<service-id> <space or dot> <action-id>' to begin or 'help' to get a list
of all available commands
E029612@live-01.ether.igrupobbva >> sync
Loading services...
Loading actions...
Loading types...
E029612@live-01.ether.igrupobbva >>
27 services loaded
846 actions loaded
72 types loaded
```


3. GETTING STARTED

3.1. Starting by programming language or runtime

ASO-LINK

El contenido se encuentra en [el siguiente documento](#).

APX

El contenido se encuentra en [el siguiente documento](#).

CELLS

El contenido se encuentra en [el siguiente documento](#).

DATAPROC

El contenido se encuentra en [el siguiente documento](#).

4. APÉNDICE - Uso de ECS-CLI y Stacker para creación de recursos y despliegues.

ECS Stacker es el servicio encargado de orquestar la creación de recursos y los despliegues. Los apartados anteriores describen como puede ser invocado de modo gráfico con la Ether Console para cada tipo de recurso.

Complementando los apartados anteriores, esta sección describe cómo se pueden crear directamente recursos con ECS-CLI así como lanzar despliegues de los mismos.

Para más información consultar: [ECS-CLI + Stacker - Provisioning](#) y [ECS-CLI + Stacker - Deployment](#)

4.1. Requisitos previos - Instalación ECS-CLI

De cara a poder crear y desplegar recursos con ECS-CLI + Stacker, es necesario realizar las siguientes tareas previas:

1. Instalación de ECS-CLI. Las instrucciones así como los ejecutables para cada plataforma están disponibles en [ECS CLI Installation](#). (NO requiere permisos de administración)

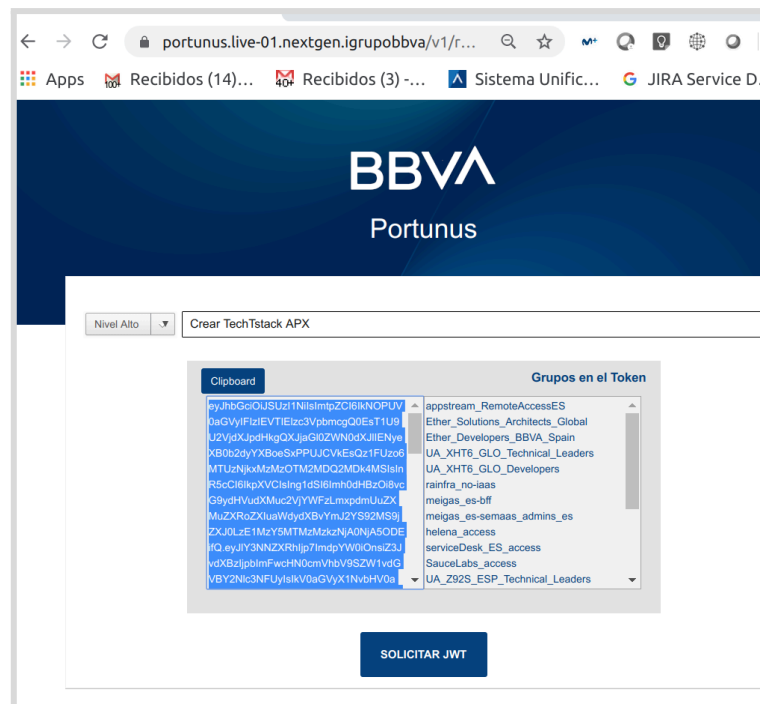
4.2. Requisitos previos - Instalación ECS-CLI

Los pasos para crear un recurso con ECS Stacker desde ECS-CLI son:

1. Desde una consola de línea de comandos, ejecutar “ecs-cli -r central.ether.igrupobbva”, pegando el token generado con portunus.

```
ecs-cli -r central.nextgen.igrupobbva
```

2. De modo automático se abre el navegador configurado por defecto posicionado en el sitio de [Portunus](#). Para generar el token, indicar que se requiere uno de “Nivel Alto”.



3. Ejecutar “sync” para actualizar las definiciones de servicios y acciones

```

ecs-cli-0.3.6
File Edit View Search Terminal Tabs Help
miguel@MP005950: ~ ecs-cli-0,3,6
#####
  e c s - c l i
Ether Cloud Services -- Command Line Interface
0.3.6
#####
Type '<service-id> <space or dot> <action-id>' to begin or 'help' to get a list
of all available commands
E029612@central.ether.igrupobbva >> sync
Loading services...
Loading actions...
Loading types...
E029612@central.ether.igrupobbva >>
  
```

4. Invocar a stacker indicando el tipo de recurso a crear y el nombre. Los tipos de recursos válidos se pueden consultar en la sección “Tech Stack” del Getting Started de [Stacker](#).

Para la creación del recurso es necesario indicar el namespace de diseño. ECS-CLI autocompleta por lo que se recomienda escribir el código de la UA de la aplicación a la que pertenece el namespace para encontrarlo más fácilmente.

```
ecs-cli-0.3.6
File Edit View Search Terminal Tabs Help
miguel@MP005950: ~ x ecs-cli-0.3.6 x
E029612@central.ether.igrupobbva >> stacker createStack --namespace hrch
```

Por ejemplo si se quiere crear un stack (recurso) de tipo apx, el comando a introducir es “stacker createStack --namespace es.hrch.app-id-6349.dsg --payload { "kind": "apx.r2r3", "name": "testapxuf" }”

```
ecs-cli-0.3.6
File Edit View Search Terminal Tabs Help
miguel@MP005950: ~ x ecs-cli-0.3.6 x
E029612@central.ether.igrupobbva >> stacker createStack --namespace es.hrch.app-id-6349.dsg --pa
yload { "kind": "apx.r2r3sss", "name": "testapxuf" }
```

5. Sobre el stack creado, hacer un scaffold para crear los repositorios asociados, build jobs, componentes de vtrack y entidad de Samuel.

```
ecs-cli-0.3.6
File Edit View Search Terminal Tabs Help
miguel@MP005950: ~ x ecs-cli-0.3.6 x miguel@MP005950: ~/git/BBVA... x
E029612@central.ether.igrupobbva >> stacker createScaffold --namespace es.hrch.app-id-6349.dsg -
-stack testapxuf --payload { "params":{ "kind": "du_online", "description": "Test to create a ne
w APX deployment unit" } }
```

Cada tipo de stack (recurso) requiere unos parámetros diferentes. Se puede consultar los parámetros en su propia documentación o invocando el siguiente comando:

```
ecs-cli-0.3.6
File Edit View Search Terminal Tabs Help
miguel@MP005950: ~ x ecs-cli-0.3.6 x miguel@MP005950: ~/git/BBVA... x
E029612@central.ether.igrupobbva >> stacker getTechStack --namespace es.hrch.app-id-6349.dsg --t
s apx.r2r3
```

Simplemente indicando el tipo de stack se detalla el schema que deben cumplir los parámetros.

6. El proceso de scaffold, salvo la fase inicial de creación del repositorio, build job etc, es asíncrona. Para consultar el estado del scaffold simplemente ejecutar:

```

ecs-cli-0.3.6
File Edit View Search Terminal Tabs Help
miguel@MP005950: ~ x ecs-cli-0.3.6 x miguel@MP005950: ~/git/BBVA... x
E029612@central.ether.igrupobbva >> stacker listScaffolds --namespace es.hrch.app-id-6349.dsg --stack testapxuf

```

Stacker muestra los diferentes pasos del pipeline de scaffold indicando para cada uno si ha funcionado correctamente o no. Además se muestra el estado global de la operación señalando si terminó correctamente o no el proceso. Se muestra además la URL del repositorio creado, el job de jenkins y el componente de vtrack. Toda la información anterior, se puede obtener también directamente a nivel de stack:

```

ecs-cli-0.3.6
File Edit View Search Terminal Tabs Help
miguel@MP005950: ~ x ecs-cli-0.3.6 x miguel@MP005950: ~/git/BBVA... x
E029612@central.ether.igrupobbva >> stacker getStack --namespace es.hrch.app-id-6349.dsg --stack testapxuf

```

```

Body:
{
  "_ac": true,
  "_createdAt": "2020-04-21T10:16:06.244Z",
  "_createdBy": "E029612",
  "_id": "a06a82a9-c0db-4ae9-97d3-40d38d57e111",
  "_locator": "//stacker.central/ns/es.hrch.app-id-6349.dsg/stacks/testapxuf",
  "_parent": "//stacker.central/ns/es.hrch.app-id-6349.dsg/stacks",
  "_type": "stacker.Stack",
  "_updatedAt": "2020-04-21T11:39:48.123Z",
  "component": "//vtrack.central/ns/es.hrch.app-id-6349.dsg/components/a06a82a9-c0db-4ae9-97d3-40d38d57e111",
  "job": "https://globaldevtools.bbva.com/piaas-es-apps-apx/job/es.hrch.app-id-6349.dsg/job/testapxuf",
  "kind": "apx.r2r3",
  "name": "testapxuf",
  "repositories": [
    {
      "links": {
        "clone": {
          "http": "https://globaldevtools.bbva.com/bitbucket/scm/es_hrch_app-id-6349_dsg/testapxuf.git",
          "ssh": "ssh://git@globaldevtools.bbva.com:7999/es_hrch_app-id-6349_dsg/testapxuf.git"
        },
        "web": "https://globaldevtools.bbva.com/bitbucket/projects/ES_HRCH_APP-ID-6349_DSG/repos/testapxuf/browse"
      },
      "master": true
    }
  ]
}

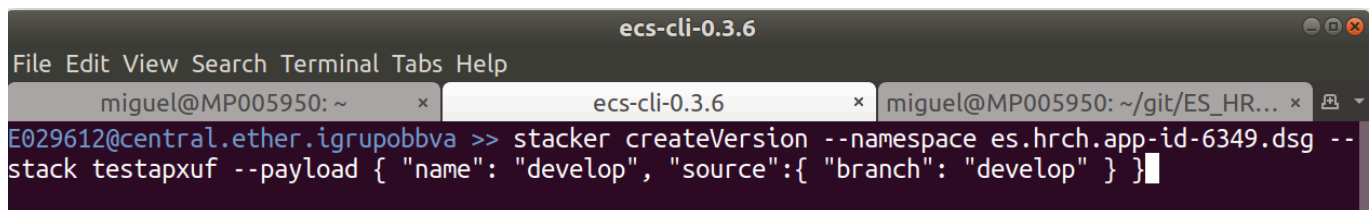
```

(Después de este punto el desarrollador hace un checkout del repositorio y empieza a implementar la funcionalidad)

4.3. Creación de una versión

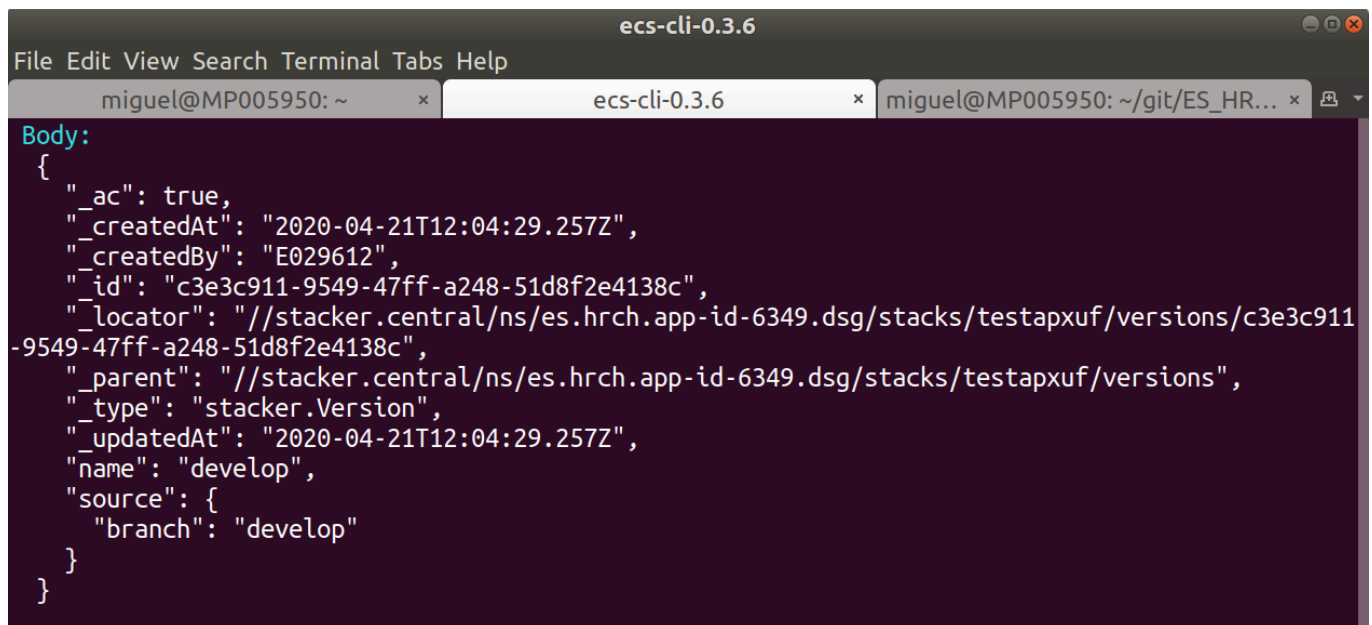
Una vez que el equipo de desarrollo tiene una versión funcionando correctamente en local, se crea una versión que será la que se desplegará en los diferentes entornos.

Simplemente es necesario indicar el stack así como la referencia al repositorio (un commit, un tag o un branch):



```
ecs-cli-0.3.6
File Edit View Search Terminal Tabs Help
miguel@MP005950: ~ x ecs-cli-0.3.6 x miguel@MP005950: ~/git/ES_HR... x
E029612@central.ether.igrupobbva >> stacker createVersion --namespace es.hrch.app-id-6349.dsg --
stack testapxuf --payload { "name": "develop", "source":{ "branch": "develop" } }
```

Se muestra la información de la versión creada. El “locator” de la misma se utiliza para lanzar el despliegue en el siguiente paso.



```
ecs-cli-0.3.6
File Edit View Search Terminal Tabs Help
miguel@MP005950: ~ x ecs-cli-0.3.6 x miguel@MP005950: ~/git/ES_HR... x
Body:
{
  "_ac": true,
  "_createdAt": "2020-04-21T12:04:29.257Z",
  "_createdBy": "E029612",
  "_id": "c3e3c911-9549-47ff-a248-51d8f2e4138c",
  "_locator": "//stacker.central/ns/es.hrch.app-id-6349.dsg/stacks/testapxuf/versions/c3e3c911-9549-47ff-a248-51d8f2e4138c",
  "_parent": "//stacker.central/ns/es.hrch.app-id-6349.dsg/stacks/testapxuf/versions",
  "_type": "stacker.Version",
  "_updatedAt": "2020-04-21T12:04:29.257Z",
  "name": "develop",
  "source": {
    "branch": "develop"
  }
}
```

1. Despliegue de una versión en un namespace (entorno)

1. En primer lugar, en este caso, se debe acceder con ecs-cli a la región donde se encuentra el namespace que representa al entorno donde se quiere desplegar.


```
ecs-cli-0.3.6
File Edit View Search Terminal Tabs Help
miguel@MP005950: ~ x ecs-cli-0.3.6 x miguel@MP005950: ~/git/... x ecs-cli-0.3.6 x
E029612@work-01.ether.igrupobbva >> stacker createDeployment --namespace es.hrch.app-id-6349.dev --payload { "
version": "//stacker.central/ns/es.hrch.app-id-6349.dsg/stacks/testapxuf/versions/c3e3c911-9549-47ff-a248-51d8
f2e4138c", "params":{ "type": "r2", "engine": "apx-online" } }
```

```
ecs-cli-0.3.6
File Edit View Search Terminal Tabs Help
miguel@MP005950: ~ x ecs-cli-0.3.6 x miguel@MP005950: ~/git/... x ecs-cli-0.3.6 x
Body:
{
  "_ac": true,
  "_createdAt": "2020-04-21T12:17:19.684Z",
  "_createdBy": "E029612",
  "_id": "216ca1cf-c315-4680-be2c-51e1e87d6067",
  "_locator": "//stacker.work-01/ns/es.hrch.app-id-6349.dev/deployments/216ca1cf-c315-4680-be2c-51e1e87d6067",
  "_parent": "//stacker.work-01/ns/es.hrch.app-id-6349.dev/deployments",
  "_type": "stacker.Deployment",
  "_updatedAt": "2020-04-21T12:17:20.347Z",
  "cancellable": false,
  "completed": false,
  "current_operation": "deployment",
  "error": false,
```

El proceso se lanza de modo asíncrono verificando en primer lugar si se cumplen los comportamientos definidos en Samuel.

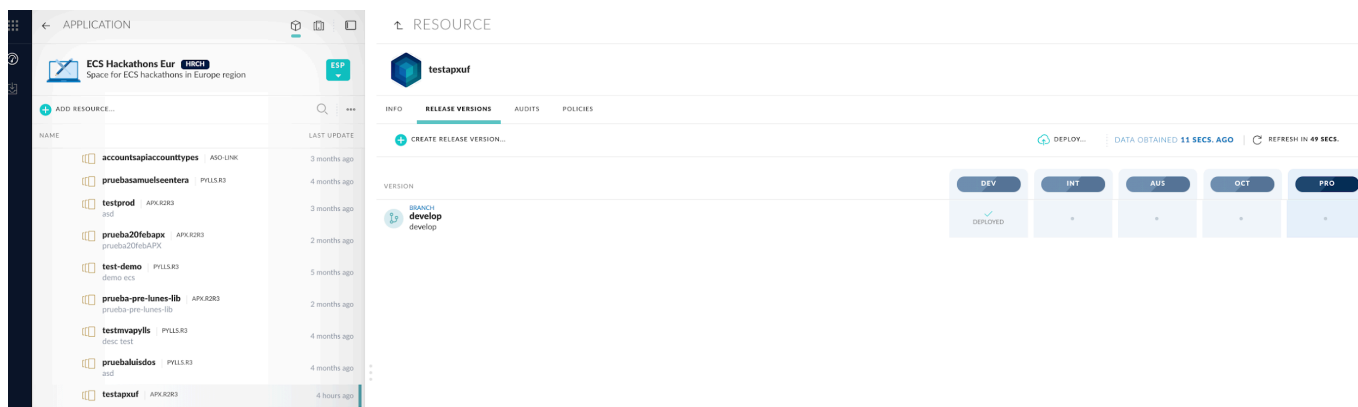
3. Se puede consultar el estado del despliegue.

```
ecs-cli-0.3.6
File Edit View Search Terminal Tabs Help
miguel@MP005950: ~ x ecs-cli-0.3.6 x miguel@MP005950: ~/git/E... x ecs-cli-0.3.6 x
E029612@work-01.ether.igrupobbva >> stacker getDeployment --namespace es.hrch.app-id-6349.dev --deployment 216ca1c
f-c315-4680-be2c-51e1e87d6067
```

```
File Edit View Search Terminal Tabs Help
miguel@MP0059... x ecs-cli-0.3.6 x miguel@MP0059... x ecs-cli-0.3.6 x
},
"stack": "//stacker.central/ns/es.hrch.app-id-6349.dsg/stacks/testapxuf",
"stage": "SCHEDULED",
"stages": [
{
"description": "Check request to deploy in progress.",
"error": false,
"operation": "deployment",
"stage": "CHECKING",
"status": "SUCCESS"
},
{
"description": "Build repository in progress.",
"error": false,
"operation": "deployment",
"stage": "GETTING ARTIFACTS",
"status": "SUCCESS"
},
{
"description": "Build repository in progress.",
"error": false,
"operation": "deployment",
"stage": "BUILDING",
"status": "SUCCESS"
},
{
"description": "Deploy repository in progress.",
"error": false,
"operation": "deployment",
"stage": "PLANNING",
"status": "SUCCESS"
},
{
"description": "Deployment scheduled to a previous environment",
"error": false,
"operation": "deployment",
"stage": "SCHEDULED",
"status": "SUCCESS"
},
{
"description": "Deployment deployed to a previous environment",
"error": false,
"operation": "deployment",
"stage": "DEPLOYED",
"status": "PENDING"
}
}
```

En este caso el despliegue ha quedado listo para la siguiente ventana de despliegue.

Lógicamente todos los recursos creados así como los despliegues lanzados desde Ether Console, ECS-CLI o con llamadas directas a el API de Stacker son visibles en ambas herramientas. En este caso, el recurso creado y desplegado vía ecs-cli se muestra en Ether Console de un modo similar a cualquier otro:



4.4. Services

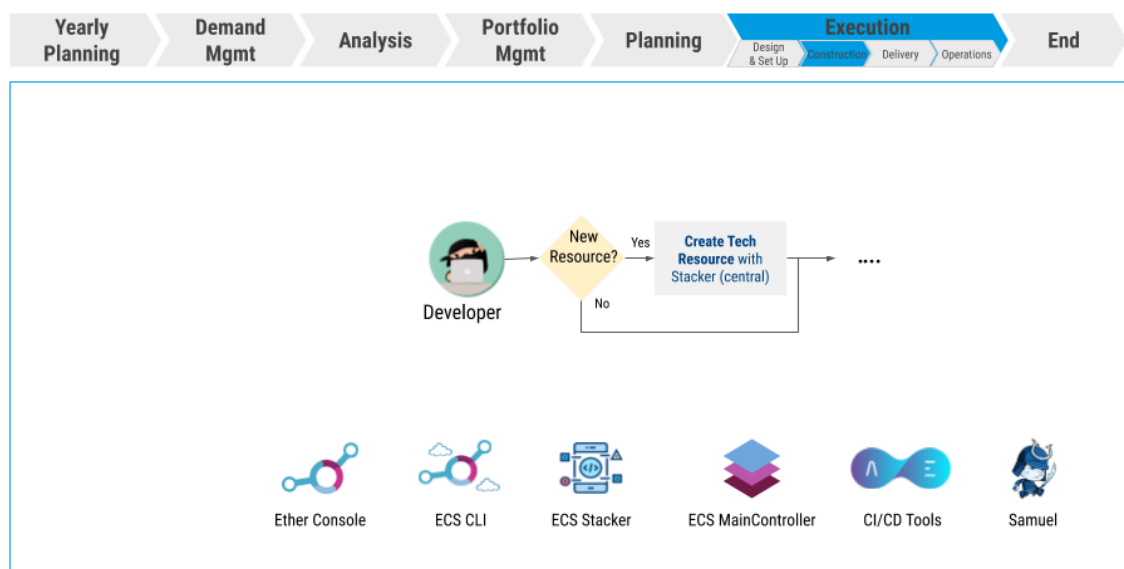
ECS Stacker

¿Qué es?

ECS Stacker es el servicio de ECS que **permite crear nuevos recursos** basados en código (aplicaciones Cells, servicios ASO, ...) así como **desplegar versiones de los mismos**.

¿En qué consiste la “creación de nuevos recursos”?

ECS in the End to End Processes



Es el proceso de provisionar un nuevo tipo de elemento (una unidad de despliegue de APX, un servicio ASO, una aplicación Cells, ...) junto con los recursos asociados (un repositorio de código fuente, un job de construcción, la entidad de Samuel asociada, etc).

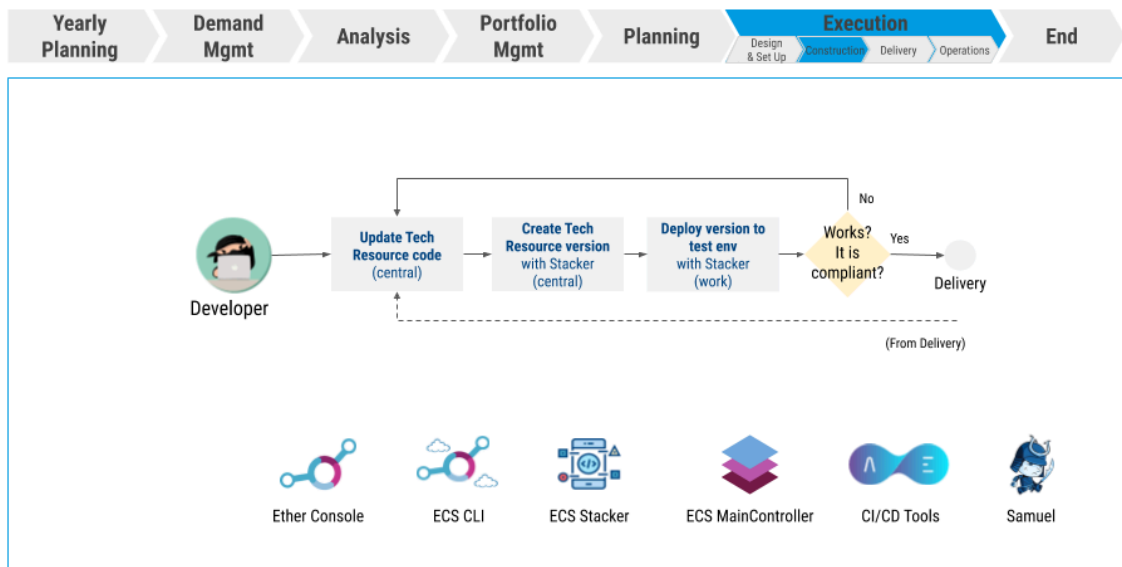
La creación de un nuevo recurso se realiza en el namespace de diseño de la región central (ver [Ether Cloud Services - Conceptos básicos](#)).

El recurso en el modelo de recursos de Stacker que representa un “elemento de desarrollo” (servicio ASO, app Cells) se denomina **stack**.

La operación para crear el repositorio asociado a un “stack” así como el job de construcción, estructura de carpetas etc se denomina “**scaffold**”. En la mayor parte de los runtimes solo se invoca a scaffold una vez al crear el recurso pero, por ejemplo, en el caso de ASO se invoca cada vez que se va a implementar un nuevo verbo en el servicio para crear la rama correspondiente de código con el esqueleto del servicio.

¿En qué consiste el “despliegue de recursos”?

ECS in the End to End Processes

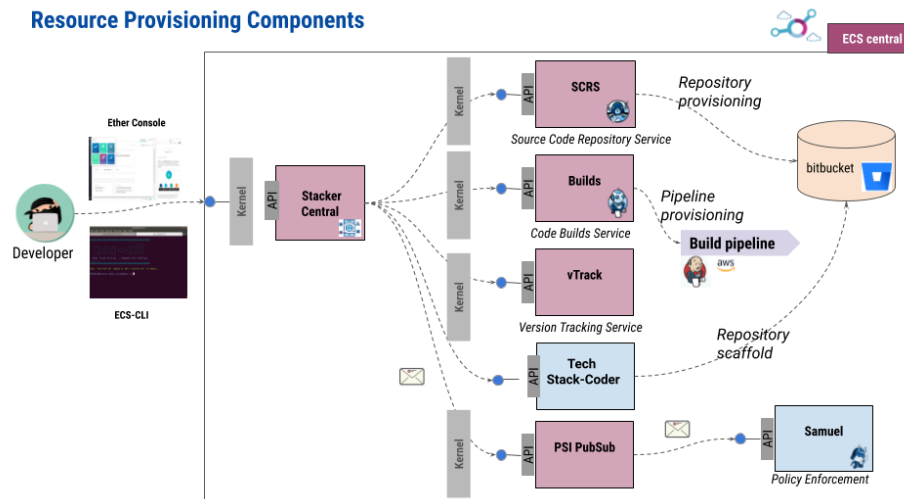


La operación de despliegue consiste en disponibilizar en un entorno (implementado en ECS como un namespace) una versión de un recurso (stack dentro de Stacker). Una versión en Stacker es simplemente un recurso relacionado con un commit, branch o tag del repositorio de código fuente.

De este modo **Stacker despliega versiones de código (stacks) en namespaces**.

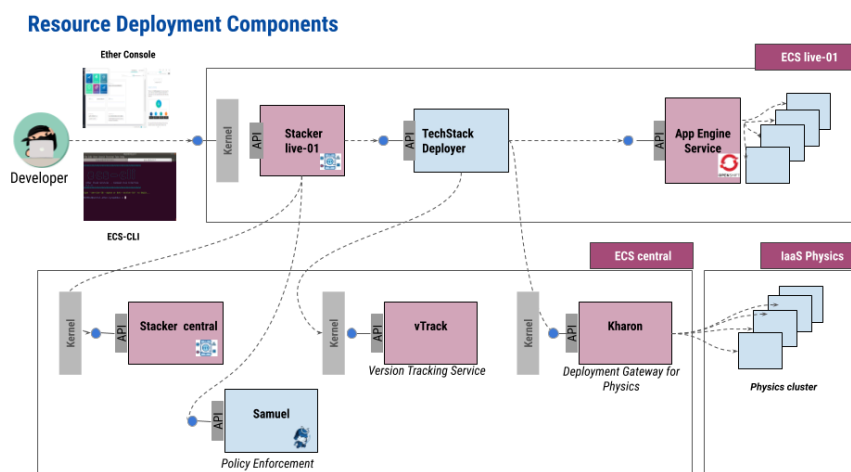
Las versiones, por estar asociadas al código fuente y, por tanto, al recurso stack, se crean como hijos del stack en el namespace de diseño en central. Los despliegues de dichas versiones, se realizan en el namespace de work o live que representa al entorno previo o a producción. (ver [Ether Cloud Services - Conceptos básicos](#)).

Técnicamente, ¿de qué partes se compone Stacker?



Stacker durante el proceso de **creación de recursos (scaffold)** invoca a los **servicios de DevOps** ([SCRS - Source Code Repository Service](#), [Builds - Code Builds Service](#), [vTrack - Version Tracking](#) y [Samuel - Policy Enforcement](#)) para crear los recursos necesarios para el proceso de desarrollo (repositorios, jobs de construcción, etc.).

Además el servicio de Stacker invoca al **TechStack de scaffold del runtime** en particular (hay uno específico para ASO, otro para APX, ...). El TechStack **realiza las tareas que son distintas por cada runtime, por ejemplo crear la estructura de carpetas** que requiere ese runtime determinado.



Para **realizar un despliegue**, el usuario de stacker indica que **versión de un stack quiere desplegar en que namespace (el cual representa entorno)**. Stacker para realizar el despliegue, en primer lugar **pregunta a Samuel si puede desplegar esa versión en ese entorno**. En caso afirmativo, invoca al TechStack de despliegue específico del tipo de runtime (hay uno específico para ASO, otro para APX, ...). El **TechStack de despliegue realiza las tareas específicas de despliegue de ese runtime** como invocar a AppEngine en los despliegues de APX R3, a kharon en los despliegues a Physics etc.

Stacker y el ecosistema de **TechStacks y servicios DevOps** trabajan de **modo asíncrono** utilizando un **tópico de PSI** para notificar el estado del despliegue.

Para más información relativa al ciclo de vida de recursos de ECS, consultar [ECS at BBVA](#)

Al final de este documento, a modo de [apéndice](#), se describen los pasos necesarios para crear recursos con stacker y desplegarlos con ECS-CLI.